

SISTEMAS DE NUMERACIÓN

INTRODUCCIÓN

El sistema de numeración binario es el más importante en los sistemas digitales, pero hay otros que también lo son. La importancia del sistema decimal radica en que se utiliza universalmente para representar cantidades fuera de un sistema digital. Ello significa que habrá situaciones en las cuales los valores decimales tengan que convertirse en valores binarios antes de que se introduzcan en el sistema digital. Por ejemplo, cuando se presiona un número decimal en una calculadora portátil (o una computadora), los circuitos que están dentro del dispositivo convierten el número decimal en un valor binario.

De igual manera, habrá situaciones en que los valores binarios de las salidas de un circuito digital tengan que convertirse a valores decimales para presentarse al mundo exterior. Por ejemplo, una calculadora (o computadora) utiliza números binarios para calcular respuestas a un problema, luego los convierte a un valor decimal antes de mostrarlos en la pantalla.

Además del binario y el decimal, otros dos sistemas de numeración encuentran amplias aplicaciones en los sistemas digitales. Los sistemas octal (base 8) y hexadecimal (base 16) se usan con la misma finalidad: ofrecer un eficaz medio de representación de números binarios grandes. Como veremos, ambos sistemas numéricos tienen la ventaja de que pueden convenirse fácilmente al y del binario.

En un sistema digital, se pueden utilizar tres o cuatro de estos sistemas de numeración al mismo tiempo, de modo que un entendimiento de la operación del sistema requiere la facultad de convertir de un sistema numérico a otro. Este capítulo mostrará al lector cómo realizar estas conversiones. Aunque algunas no se usarán de inmediato en nuestro estudio de los sistemas digitales, se necesitarán cuando comencemos a estudiar los microprocesadores. Este capítulo presentará, asimismo, algunos de los códigos binarios que se utilizan para representar diversos tipos de información. Estos códigos binarios usarán 1 y 0, pero en una forma que difiere un tanto del sistema numérico binario.

CONVERSIONES DE BINARIO A DECIMAL

Como vimos en el capítulo 1, el sistema de numeración binario es un sistema posicional donde cada dígito binario (bit) tiene un valor basado en su posición relativa al LSB. Cualquier número binario puede convertirse a su equivalente decimal, simplemente sumando en el número binario las diversas posiciones que contenga un 1. Para ilustrar lo anterior consideremos el siguiente ejemplo:

$$\begin{array}{rcccccl}
 1 & 1 & 0 & 1 & 1 & = & \text{(binario)} \\
 2^4 & +2^3 & +0 & +2^1 & +2^0 & = & 16+8+2+1 \\
 & & & & & = & 24_{10} \quad \text{(decimal)}
 \end{array}$$

Ahora daremos otro ejemplo con números mayor de bits:

$$\begin{array}{rcccccccccl}
 1 & 0 & 1 & 1 & 0 & 1 & 0 & 1 & = \\
 2^7 & +0 & +2^5 & +2^4 & +0 & +2^2 & +0 & +2^0 & = & 181_{10}
 \end{array}$$

Nótese que el procedimiento consiste en determinar los valores (es decir, las potencias de 2) de cada posición de bit que contenga un 1 y luego sumarlos. Nótese también que el MSB tiene un valor de 2^7 a pesar de que es el octavo bit; esto se debe a que el LSB es el primer bit y tiene un valor de 2^0 .

CONVERSIONES DE DECIMAL A BINARIO

División repetida otro método emplea la división repetida por 2. La conversión, que se ilustra a continuación para 2510, requiere dividir repetidamente el número decimal entre 2 y que se escriban los residuos después de cada división hasta que se obtenga un cociente 0. Nótese que el resultado binario se obtiene al escribir el primer residuo como el LSB y el último como el MSB.

$$\begin{array}{lcl}
 & 0,5 \times 2 & \\
 25/2 = 12 + \text{residuo de } 1 & \xrightarrow{\hspace{1.5cm}} & \\
 12/2 = 6 + \text{residuo de } 0 & \xrightarrow{\hspace{1.5cm}} & \\
 6/2 = 3 + \text{residuo de } 0 & \xrightarrow{\hspace{1.5cm}} & \\
 3/2 = 1 + \text{residuo de } 1 & \xrightarrow{\hspace{1.5cm}} & \\
 1/2 = 0 + \text{residuo de } 1 & \xrightarrow{\hspace{1.5cm}} & \\
 & \text{MSB} \quad 25_{10} = & 1 \quad 1 \quad 0 \quad 0 \quad 1_2
 \end{array}$$

Este proceso también se puede emplear para convertir de decimal a cualquier otro sistema numérico.

Si se utiliza una calculadora para efectuar las divisiones entre 2, los residuos se pueden determinar al observar si el cociente tiene parte fraccionaria o no. Por ejemplo, la calculadora produciría $25/2 = 12.5$. El .5 indica que hay un residuo

de 1. Asimismo, la calculadora nos daría $12/2 = 6.0$, que indica un residuo de 0. A continuación se muestra otro ejemplo tal y como se obtendría utilizando una calculadora.

$$\begin{aligned}
 37/2 &= 18.5 + \text{residuo de } 1 \\
 18/2 &= 9 + \text{residuo de } 0 \\
 9/2 &= 4.5 + \text{residuo de } 1 \\
 4/2 &= 2 + \text{residuo de } 0 \\
 2/2 &= 0 + \text{residuo de } 0 \\
 1/2 &= 0.5 + \text{residuo de } 1 \quad \text{MSB}
 \end{aligned}$$

Por tanto, $37_{10} = 100101_2$

SISTEMA DE NUMERACIÓN OCTAL

El sistema de numeración octal es muy importante en el trabajo que se realiza en una computadora digital. Este tiene una base de *ocho*, lo cual significa que tiene ocho posibles dígitos: 0, 1, 2, 3, 4, 5, 6 y 7. Así, cada dígito de un número octal puede tener cualquier valor del 0 al 7. Las posiciones de los dígitos en un sistema octal tienen los siguientes valores:

8^4	8^3	8^2	8^1	8^0	8^{-1}	8^{-2}	8^{-3}	8^{-4}	8^{-5}
-------	-------	-------	-------	-------	----------	----------	----------	----------	----------

↓
punto octal

Conversión de octal a decimal Por tanto, un número octal puede convertirse fácilmente a su equivalente decimal multiplicando cada dígito octal por su valor posicional. Por ejemplo:

$$\begin{aligned}
 372_8 &= 3 \times (8^2) + 7 \times (8^1) + 2 \times (8^0) \\
 &= 3 \times 64 + 7 \times 8 + 2 \times 1 \\
 &= 250_{10}
 \end{aligned}$$

Conversión de decimal a octal Un entero decimal se puede convertir a octal con el mismo método de división repetida que se usó en la conversión de decimal a binario, pero con un factor de división de 8 en lugar de 2. A continuación se muestra un ejemplo de esto.

$$\begin{aligned}
 266/8 &= 33 + \text{residuo de } 2 \\
 33/8 &= 4 + \text{residuo de } 1
 \end{aligned}$$

$$4/8 = 0 + \text{residuo de } 4 \xrightarrow{\quad\quad\quad} 266_{10} = 412_8$$

Nótese que el primer residuo se transforma en el dígito menos significativo (LSD) del número octal y el último residuo se convierte en el dígito más significativo (MSD).

Si se emplea una calculadora para realizar las divisiones del proceso anterior, el resultado incluirá una fracción decimal en lugar de un residuo. Sin embargo el residuo puede obtenerse multiplicando la fracción decimal por 8. Por ejemplo, $266/8$ produce 33.25. El residuo se convierte en $0.25 \times 8 = 2$. En forma similar, $33/8$ dará como resultado 4.125 y el residuo se transforma en $0.125 \times 8 = 1$.

Conversión de octal a binario La ventaja principal del sistema de numeración octal es la facilidad con que se puede realizar la conversión entre números binarios y octales. La conversión de octal a binario se lleva cabo convirtiendo cada dígito octal en su equivalente binario de 3 bits. Los ocho dígitos posibles se convierten como se indica en la tabla (2-1):

Dígito Octal	0	1	2	3	4	5	6	7
Equivalente binario	000	001	010	011	100	101	110	111

Por medio de estas conversiones, cualquier número octal se convierte a binario, convirtiéndolo de manera individual. Por ejemplo, podemos convertir 472_8 a binario de la siguiente manera:

$$\begin{array}{ccc} 4 & 7 & 2 \\ \downarrow & \downarrow & \downarrow \\ 100 & 111 & 010 \end{array}$$

Por tanto, el número octal 472 es equivalente a binario 100111010. Para dar otro ejemplo, consideremos convertir el número 5431_8 a binario:

$$\begin{array}{cccc} 5 & 4 & 3 & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 101 & 100 & 011 & 001 \end{array}$$

$$\text{Por lo tanto, } 5431_8 = 101100011001_2$$

Conversión de binario a octal La conversión de enteros binarios a octales es simplemente la operación inversa del proceso anterior. Los bits del número binario se agrupan en conjuntos de tres comenzando por el LSB. Luego, cada grupo se convierte a su equivalente octal (tabla 2-1). Para ilustrarlo, consideremos la conversión de 100111010_2 a octal.

1	0	0	1	1	1	0	1	0
			4		7		2	₈

Algunas veces el número binario no tendrá grupos pares de 3 bits. Es esos casos, podemos agregar uno o dos ceros a la izquierda del MSB del número binario a fin de completar el último grupo. Esto se ilustra a continuación para el número binario 11010110 .

0	1	1	0	1	0	1	1	0
			3		2		6	₈

Nótese que se colocó un 0 a la izquierda del MSB para producir grupos pares de 3 bits.

Utilidad del sistema octal La facilidad con que se pueden hacer conversiones entre el sistema octal y el binario hace que el sistema octal sea atractivo como un medio “taquigráfico” de expresión de números binarios grandes. En computación, son comunes los números binarios con 64 bits. Estos, como veremos, no siempre representan una cantidad numérica, sino que a menudo son cierto tipo de código que conlleva información no numérica. En las computadoras, los números binarios podrían representar (1) datos numéricos reales; (2) números correspondientes a una localidad (dirección) en la memoria; (3) un código de instrucción; (4) un código que representa caracteres alfabéticos y otros no numéricos, o (5) un grupo de bits que representan las condiciones en que se encuentran los dispositivos internos o externos a la computadora.

Cuando trabajamos con una gran cantidad de números binarios de muchos bits, es más conveniente y eficaz escribirlos en octal y no en binario. Sin embargo, recordemos que los circuitos y sistemas digitales trabajan estrictamente en binario; usamos el sistema octal sólo por conveniencia de los operadores del sistema.

SISTEMA DE NUMERACIÓN HEXADECIMAL

El sistema hexadecimal emplea la base 16. Así, tiene 16 posibles símbolos digitales. Utiliza los dígitos del 0 al 9 más las letras A, B, C, D, E y F como sus 16 símbolos digitales. Nótese que cada dígito hexadecimal representa un grupo de cuatro dígitos binarios. Es importante recordar que los dígitos hex (abreviatura de hexadecimal) de A a F son equivalentes a los valores decimales de 10 al 15.

Conversión de hexadecimal a decimal Un número hex se puede convertir a su equivalente decimal utilizando el hecho de que cada posición de los dígitos hex tiene un valor que es una potencia de 16. El LSD tiene un valor de $16^0 = 1$; el siguiente dígito en secuencia tiene un valor de $16^1 = 16$; el siguiente tiene un valor de $16^2 = 256$ y así sucesivamente. El proceso de conversión se demuestra en los ejemplos que siguen:

$$\begin{aligned} 356_{16} &= 3 \times 16^2 + 5 \times 16^1 + 6 \times 16^0 \\ &= 768 + 80 + 6 \\ &= 854_{10} \end{aligned}$$

$$\begin{aligned} 2AF &= 2 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 \\ &= 512 + 160 + 15 \\ &= 687_{10} \end{aligned}$$

Nótese que en el segundo ejemplo el valor 10 se sustituyó por A y el valor 15 por F en la conversión al sistema decimal.

A manera de ejercicio, verifique que $1BC2_{16}$ es igual a 7106_{10} .

Conversión de decimal a hexadecimal Recuerde que efectuamos la conversión de decimal a binario por medio de la división repetida entre 2 y de decimal a octal por medio de la división repetida entre 8. De igual manera, la conversión de decimal a hexadecimal se puede efectuar por medio de la división repetida entre 16. Los dos ejemplos siguientes ilustrarán esto.

convertir 423_{10} a hexadecimal

$$\begin{array}{ll} 423/16 = 26 + \text{residuo de } 7 & \longrightarrow 7 \\ 26/16 = 1 + \text{residuo de } 10 & \longrightarrow A \\ 4/16 = 0 + \text{residuo de } 4 & \longrightarrow 4 \end{array}$$

$$423_{10} = 1^a7_{16}$$

convertir 214_{10} a hexadecimal

Solución:

$$\begin{array}{lcl} 214/16 = 13 + \text{residuo de } 6 & \longrightarrow & 6 \\ 13/16 = 0 + \text{residuo de } 13 & \longrightarrow & D \end{array}$$

$$214_{10} = D6_{16}$$

Nótese una vez más la forma en que los residuos de los procesos de división forman los números del dígito hexadecimal. Nótese también que los residuos mayores de 9 se representan por las letras de la A a la F.

Si se emplea una calculadora para realizar las divisiones en el proceso de conversión, los resultados incluirán una fracción decimales lugar de un residuo. El residuo se puede obtener multiplicando la fracción por 16. Para ilustrar lo anterior en el segundo ejemplo la calculadora habría producido el resultado.

$$214/16 = 13.375$$

$$\text{El residuo se convierte en } (0.375) \times 16 = 6.$$

Conversión de hexadecimal a binario Al igual que el sistema de numeración octal, el sistema hexadecimal se usa principalmente como método “taquigráfico” en la representación de números binarios. Es una tarea relativamente simple la de convertir un número hexadecimal en binario. Cada dígito hexadecimal se convierte en su equivalente binario de 4 bits. Esto se ilustra a continuación para el número $9F2_{16}$

$$\begin{array}{cccc} 9 & & F & & 2 \\ & 1001 & 1111 & & 0010 \\ 9F2_{16} = & & & & \\ & = & 1001111100101_2 \end{array}$$

A manera de ejercicio verifique que $BA6_{16} = 101110100110_2$

Conversión de binario a hexadecimal Esta conversión es exactamente la operación inversa del proceso anterior. El número binario se agrupa en

conjuntos de cuatro bits y cada grupo se convierte a su dígito hexadecimal equivalente. Cuando es necesario se añaden ceros para completar un grupo de cuatro bits.

$$\begin{array}{rcc} 1110100110_2 & = & 0011 \quad 1010 \quad 0110 \\ & & 3 \quad \quad A \quad \quad 6 \\ & = & 3A6_{16} \end{array}$$

A fin de efectuar estas conversiones entre hexadecimal y binario es necesario conocer los números binarios de 4 bits (0000 – 1111) y sus dígitos hexadecimales equivalentes. Una vez que se conozcan bien, se pueden realizar rápidamente las conversiones sin tener que realizar operaciones. Esta es la razón por la cual los sistemas hexadecimales y octal son tan útiles en la representación de números binarios grandes.

convertir el número decimal 378 a un número binario de 16 bits convirtiéndolo primero a un número hexadecimal.

$$\begin{array}{lcl} 378/16 = 23 + \text{residuo de } 10 & \longrightarrow & 10 \\ 23/16 = 1 + \text{residuo de } 10 & \longrightarrow & 7 \\ 1/16 = 0 + \text{residuo de } 1 & \longrightarrow & 1 \end{array}$$

Así pues, $378_{10} = 17A_1$. Este número hexadecimal se puede convertir con facilidad al número binario 000101111010. Para terminar, se puede expresar 378_{10} como un número binario de 16 bits al añadir cuatro ceros:

$$378_{10} = 0000\ 0001\ 0111\ 1010_2$$

convertir B2F₁₆, en octal

Es más fácil convertir primero de hexadecimal a binario y luego a octal

B2F ₁₆ =	1011	0010	1111	{convierta a binario}
=	101	100	101	111 {agrupar en 3 bits}
=	5	4	57	{convierta a octal}

Resumen de Conversiones

Al llegar a este punto, probablemente su cabeza le está dando vueltas conforma usted trata de grabarse con claridad todos

estos sistemas de numeración: binario, decimal, octal, hexadecimal y todas las diferentes conversiones de uno a otro. Quizá usted no lo crea, pero con el tiempo, conforme emplea éstos sistemas cada vez más, los llegará conocer a fondo. Por el momento, el siguiente resumen le ayudará a efectuar las diferentes conversiones:

1. Al convertir de binario (u octal o hexadecimal) a decimal, utilice el método de tomar la suma ponderada de cada posición de dígito.
2. Al convertir de decimal a binario (u octal o hexadecimal), utilice el método de la división repetida entre 2 (o entre 8 o 16) y registrar residuos.
3. Al convertir de binario a octal (o hexadecimal), agrupe los bits en grupos de tres (o cuatro) y convierta cada grupo al dígito octal (o hexadecimal) correcto.
4. Al convertir de octal (o hexadecimal) a binario, convierta cada dígito al equivalente de tres bits (o de 4 bits).
5. Al convertir de octal a hexadecimal (o viceversa), primero convierta a binario; luego convierta el binario al sistema deseado de numeración.

CÓDIGO BCD (Codificación Binaria Directa)
--

Cuando se representan números, letras o palabras por medio de un grupo especial de símbolos, se dice que se encuentran codificados, y al grupo de símbolos se le llama código. Probablemente uno de los más conocidos es el Morse, en el cual las series de puntos y rayas representan letras del alfabeto.

Hemos visto que cualquier número decimal se puede representar por medio de un número binario equivalente. El grupo de ceros y unos contenidos en el número binario se puede considerar como un código que representa el número decimal. Cuando un número decimal se representa con su número binario equivalente, recibe el nombre de codificación binaria directa.

Todos los sistemas digitales emplean alguna forma de números binarios para su operación interna, aunque el mundo externo es de naturaleza decimal. Ello significa que las conversiones entre los sistemas decimal y binario se llevan a cabo con frecuencia. Hemos visto que estas conversiones pueden ser largas y complicadas si se trabaja con números grandes. Por esta razón, se emplea, en ciertas situaciones, un medio de codificación de números decimales que combina características de los sistemas decimal y binario.

Código decimal codificado en binario Si cada dígito de un número decimal se representa con su equivalente binario, el resultado es un código llamado decimal codificado en binario (en los sucesivos se abreviará BCD). Ya que el mayor dígito decimal es el 9, se necesitan 4 bits para codificar cada dígito (el código binario correspondiente a 9 es 1001).

Para ilustrar el código BCD tomemos un número decimal como 874. Cada dígito se cambia por su equivalente binario de la siguiente manera:

8	7	4	(decimal)
1000	0111	0100	(BCD)

Para poner otro ejemplo, cambiemos 943 por su representación en el código BCD:

9	4	3	(decimal)
1001	0100	0011	(BCD)

Una vez más, cada dígito decimal se cambia por su equivalente binario directo. Nótese que siempre se usan 4 bits por cada dígito.

El código BCD, por tanto, representa cada dígito del número decimal por medio de un número binario de 4 bits. De hecho, sólo se utilizan números

binarios de 4bits del 0000 al 1001. El código BCD no usa los números 1010, 1011, 1100, 1101, 1110, 1111. En otras palabras, sólo se emplean 10 de los 16 posibles grupos del código binario de 4 bits. Si alguno de estos números “prohibidos” de cuatro bits aparece en una máquina que emplee el código BCD es una indicación, por lo general, de que se ha cometido un error.

Convertir 0110100000111001 (BCD) a su equivalente decimal.

divida el número BCD en grupos de 4 bits y convierta cada uno a decimal

0110 1000 0011 1001

6 8 3 9

Convertir 011111000001 (BCD) a su equivalente decimal.

0111 1100 0001

7 4 1

el grupo de código prohibido
indica error en el número BCD

Comparación de BCD y binario Es importante entender que el BCD no es otro sistema como el binario, el octal, el decimal y el hexadecimal. Es, de hecho, el sistema decimal con cada dígito codificado en su equivalente binario. También es importante comprender que un número BCD no es lo mismo que un número binario directo. Este toma un número decimal completo y lo representa en binario; el código BCD convierte cada dígito decimal a binario de manera individual. Para ilustrar lo anterior, tomemos el número 137 y comparemos sus códigos binario directo y BCD:

$137_{10} = 10001001_2$ (binario)

$137_{10} = 0001 \ 0011 \ 0111$ (BCD)

El código BCD requiere 12 bits, en tanto que el binario directo requiere únicamente 8 para representar el número 137. El código BCD requiere más bits que el binario directo para representar números decimales de más de un dígito. Ello se debe a que el BCD no emplea todos los grupos posibles de 4 bits como se indicó antes y, por tanto, es un poco ineficiente.

La ventaja principal del código BCD es la relativa facilidad de conversión a y desde decimal. Sólo se requiere recordar los grupos de código de 4 bits para los dígitos decimales del 0 al 9. Esta facilidad de conversión es especialmente importante desde el punto de vista de la circuitería (hardware) que se puede utilizar, ya que en un sistema digital son los circuitos lógicos los que efectúan las conversiones a y desde decimal.

CÓDIGO GRAY

El *código Gray* pertenece a una clase de códigos llamada *códigos de cambio mínimo*, en los cuales sólo un bit del grupo de código cambia cuando pasa de una etapa a la siguiente. El código Gray es un código *sin valor*, es decir, las posiciones de los bits en los grupos de código no tienen ningún valor específico. Debido a esto el código no se ajusta a las operaciones aritméticas sino que halla aplicación en dispositivos de entrada y salida y en algunos tipos de convertidores de analógico a digital.

Si examinamos los grupos en código Gray para cada número decimal, se puede observar que al pasar de un número decimal cualquiera al siguiente, sólo cambia un bit del código Gray. Por ejemplo, al pasar de 3 a 4, el código Gray cambia de 0010 a 0110 y sólo varía el segundo bit de la izquierda. Al pasar de 14 a 15 los bits del código Gray cambian de 1001 a 1000 y sólo varía el último bit. Esta es la característica principal del código Gray. Comparemos esto con el código binario, donde incluso pueden cambiar todos los bits al ir de un paso al otro.

El código Gray se usa en situaciones en las cuales otros códigos, como el binario, podrían producir resultados erróneos o ambiguos durante estas transiciones en las cuales cambia más de un bit de código. Esto es, al aplicar el código binario y pasar de 0111 a 1000 se requiere que los cuatro bits cambien simultáneamente. Dependiendo del dispositivo o circuito que generen los bits, puede haber una diferencia significativa en los tiempos de transición de los diferentes bits. Si esto es así, la transición de 0111 a 1000 podría producir uno o más estados intermedios. Por ejemplo, si el bit más significativo cambia más rápido que el resto, ocurrirán las siguientes transiciones:

0	1	1	1	—	decimal 7
1	1	1	1		(código erróneo)
1	0	0	0		(decimal 8)

La aparición de de 111 sólo es momentánea pero podría producir una operación errónea de los elementos que son controlados por los bits. Obviamente, el uso del código Gray eliminaría ese problema, ya que sólo ocurre un cambio de bit por transición y no puede haber jerarquías.

CÓDIGOS ALFANUMÉRICOS

Además de los datos numéricos, una computadora debe ser capaz de manejar información no numérica. En otras palabras, una computadora debe reconocer códigos que representan letras del alfabeto, signos de puntuación y otros caracteres especiales, además de los números. Estos códigos se denominan *códigos alfanuméricos*. Un código completo de este tipo puede incluir 26 letras minúsculas, 26 mayúsculas, 10 dígitos, 7 signos de puntuación y entre 20 y 40 caracteres más como +, /, %, * y otros similares. Podemos afirmar que un código alfanumérico representa a todos los caracteres y funciones diferentes que se encuentran el teclado estándar de una máquina de escribir (o de una computadora)

Código ASCII El código alfanumérico más utilizado en la mayoría de las microcomputadoras y minicomputadoras y en muchos sistemas grandes de cómputo es el Código Americano Estándar para el Intercambio de Información (ASCII, siglas de American Stándar Code for Information Interchange). El código ASCII usa 7 bits, por tanto tiene $2^7 = 128$ grupos de posibles códigos. Esta cantidad es más que suficiente para representar todos los caracteres de un teclado estándar y las funciones de control como < RETURN > (retroceso de carro) y < LINEFEED > (cambio de línea).

Ejemplo

El siguiente es un mensaje codificado en ASCII ¿Cuál es el mensaje?

1001000 1000101 1001100 1010000

Primero se convierte cada código de 7 bits en su equivalente hexadecimal. Los resultados son:

48 45 4C 50

Ahora determine el símbolo que representa cada uno. Los resultados son:

HELP (ayuda)

El código ASCII se emplea para transferir información alfanumérica entre una computadora y dispositivos de entrada/salida como terminales o impresoras. Una computadora también lo emplea internamente para guardar la informa-

ción que el operador proporciona mediante el teclado. El siguiente ejemplo ilustra lo anterior.

Un operador está escribiendo un programa en BASIC en el teclado de cierta microcomputadora. Ésta convierte la pulsación de cada tecla en su código ASCII y lo guarda en la memoria. Determine los códigos que serán colocados en la memoria cuando el operador teclee la siguiente instrucción en BASIC:

GOTO 25

El primer paso consiste en localizar cada símbolo (incluido el espacio en blanco) y anotar su código ASCII.

G	1000111
O	1001111
T	1010100
O	1001111
(espacio)	0100000
2	0110010
5	0110101

MÉTODO DE PARIEDAD PARA LA DETECCIÓN DE ERRORES

La operación más frecuente realizada por los sistemas digitales es el movimiento de datos y códigos binarios de una localidad a otra. He aquí algunos ejemplos de esta operación:

- ☒ Transmisión de una voz digitalizada sobre un enlace de microondas.
- ☒ El almacenamiento y la recuperación de datos de los dispositivos externos de memoria, como cintas y discos magnéticos.
- ☒ La transmisión de información de una computadora a una terminal remota u otra computadora.

Sin importar qué tipo de información se transmite de un dispositivo (transmisor) a otro (receptor), existe siempre la posibilidad de que se presenten errores que le impidan al receptor recibir la misma información que envió el transmisor. La causa más importante de errores es la transmisión de ruido eléctrico.

La mayor parte del equipo digital moderno está diseñado para estar relativamente libre de error, y la probabilidad de que ocurran errores es muy baja. Sin embargo los sistemas digitales a menudo transmiten miles, incluso millones, de bits por segundo, por lo que, incluso una tasa muy pequeña de ocurrencia, si puede provocar algún molesto error ocasional, si no es que hasta desastroso. Por

esta razón muchos sistemas digitales emplean algún método para detectar (y en caso corregir) errores. Uno de los esquemas más sencillos y de mayor uso para la detección de errores es el **método de la pariedad**.

El **bit de pariedad** un bit de pariedad es un bit extra que se agrega a un grupo de código que se transfiere de una localidad a otra. El bit de pariedad es un 0 o un 1, según el número de unos que haya en el grupo de código. Para esto se emplean dos métodos diferentes.

En el Método de Pariedad PAR: el valor del bit de pariedad se escoge de manera que el número total de unos en el grupo de código (incluido el bit de pariedad) sea un número par.

Por ejemplo: 1000011 = 11000011 (agregándole el bit de pariedad 1)

Si el grupo de código contiene un número par de unos, al bit de pariedad se le asigna un valor de 0.

Por ejemplo: 10010011 = 010010011 (agregándole el bit de pariedad 0)

En el Método de Pariedad IMPAR: se utiliza exactamente de la misma forma, excepto que se escoge el bit de pariedad para que el número total de unos (incluyendo al bit de pariedad) sea un número impar.

Por ejemplo: 1000001 = 11000001

Sin importar si se utiliza la pariedad par o la impar, el bit de pariedad se convierte en una parte real de la palabra del código. Por ejemplo, la adición de un bit de pariedad al código ASCII de 7 bits produce un código de 8 bits.

El bit de pariedad se emplea para detectar cualquier error en un solo bit que ocurra durante la transmisión de un código de una localidad a otra (por ejemplo, de una computadora a una terminal). Así supongamos que el carácter “A” se está transmitiendo y se emplea pariedad impar. El código transmitido sería: 11000001.

Cuando el circuito receptor capte este código, verificará que el código contenga un número impar de unos (incluyendo al bit de pariedad). Si es así, el receptor supondrá que el código se ha recibido adecuadamente. Ahora bien, supongamos que debido a algún ruido o mal funcionamiento el receptor en realidad recibe el siguiente código:

11000000

El receptor notará que este código tiene un número par de unos. Esto le indica al receptor que debe haber un error en el código, ya que presumiblemente el transmisor y el receptor han acordado utilizar la paridad impar. Sin embargo, no hay forma de que el receptor pueda indicar cuál bit contiene el error, ya que no sabe qué código se supone que es.

Debe aclararse que este método de paridad no funcionaría si dos bits contuvieran algún error, ya que dos errores no cambiarían la condición de “impar” o “par” del número de unos que hay en el código. En la práctica, el método de paridad se emplea sólo en situaciones donde la probabilidad de que se cometa un solo error es muy baja y la probabilidad de errores dobles es esencialmente nula.

Cuando se emplea el método de paridad, el transmisor y el receptor deben ponerse de acuerdo con anticipación, sobre el tipo de paridad que se utilizará: par o impar. No existe ninguna ventaja de una sobre la otra, aunque parece que la que más se emplea es la paridad par.

Este proceso recibe el nombre de **verificación de la paridad de los datos**. En caso de que detecte un error, el receptor puede enviar un mensaje al transmisor solicitándole que vuelva a transmitir el último conjunto de datos. El procedimiento exacto que se sigue cuando se detecta un error depende del diseño particular del sistema.

Ejemplo:

Un transmisor envía a un receptor datos codificados en ASCII con un bit de paridad par. Muestre los códigos que el transmisor enviará cuando transmita el mensaje “HELLO” (HOLA).

Solución:

Primero busque los códigos ASCII de todos los caracteres que conforman el mensaje. Después, para cada código, cuente el número de unos. Si este número es par, agregue un cero. Si es impar, agregue un uno. Todos los códigos de 8 bits tendrán un número par de unos (incluyendo el bit de paridad).

H-	0	1001000
E-	1	1000101
L-	1	1001100
L-	1	1001100
O-	1	1001111